Digital Signal Processing Exercise Sheet 5

Nyquist

Script: Chapter 5 (Sampling Theorem, Aliasing, Reconstruction)

Learning goals

- Predict alias frequencies with the folding rule and the folding diagram; determine minimum sampling rates, including band-pass (under-)sampling.
- Observe aliasing of the knock resonances on real (simulated) microcontrollers and verify the predictions quantitatively.
- Compare zero-order hold, linear and sinc reconstruction.
- Design an anti-aliasing filter: trade-off between filter order, sampling rate and passband loss.

Setup: JSLinux

From this sheet on, the lab exercises also run in JSLinux, a Linux system in the browser. Sheets 0–4 ran entirely in Wokwi; if you have not done the JSLinux bonus exercises of Sheets 0–2, set it up now (about 10 min):

1. Open `https://bellard.org/jslinux`, start the *x86 Alpine Linux* (console) virtual machine and check the compiler with `gcc -version` (or `tcc -v`).
2. Upload the course headers `engine_signals.h`, `asciiplot.h`, `csvio.h` (`code/common/`) and the files of the exercise (e.g. `code/jslinux/sheet05/`) with the upload button below the terminal; check with `ls` that they arrived. Headers and program must be in the same directory.
3. Compile and run, e.g.

```
gcc -O2 -o recon recon.c -lm
./recon
```

4. The VM forgets all files when the page is reloaded: keep your sources on your own computer and upload them again. Text from Wokwi (Serial Monitor) gets into the VM with `cat > data.csv`, paste, `Ctrl-D`; `export_file data.csv` downloads a file from the VM to your computer.

Background

Sampling with $f_s = 1/T_s$ makes all frequencies $f + k f_s$, $k \in \mathbb{Z}$, indistinguishable. A real tone of frequency f therefore appears at the *alias frequency*

$$f_a = |\, f - k f_s|, \qquad k = \mathrm{round}(f/f_s), \qquad 0 \le f_a \le f_s/2.$$

A band-limited signal ($X(f) = 0$ for $|f| \ge B$) is uniquely determined by its samples if $f_s > 2B$ and can be reconstructed by sinc interpolation $x(t) = \sum_n x[n]\,\mathrm{sinc}\big((t - nT_s)/T_s\big)$, $\mathrm{sinc}(u) = \sin(\pi u)/(\pi u)$. Everything above $f_s/2$ has to be removed *before* sampling by an analog anti-aliasing (AA) filter.

In our virtual engine the knock sensor has resonances at 6.5 kHz and 10.5 kHz ($\tau = 0.8$ ms) and valve impacts at 8 kHz ($\tau = 0.4$ ms). The `engine-sim` chip updates its analog outputs every 20 µs (attribute `fs_khz = 50`) and holds the value in between: the chip itself is a sampled system with zero-order hold.

Exercise 5.1 Aliasing by hand

PAPER

a) Compute the alias frequencies of the components 6.5 kHz, 8 kHz, 10.5 kHz and of a (hypothetical) third knock mode at 15 kHz for $f_s = 5$, 8.33, 16.67, 25 and 50 kHz. (8.33 kHz is close to the maximum rate of the Uno ADC.)

b) Draw the folding diagram (f_a over f, 0 kHz to 16 kHz) for $f_s = 8.33$ kHz and mark the four components. Which two components can no longer be separated after sampling?

c) What is the minimum sampling rate for a knock signal with components up to 15 kHz? A 4th-order Butterworth AA filter with $f_c = 15$ kHz ($|H| = 1/\sqrt{1 + (f/f_c)^8}$) is used. Which f_s is needed so that every frequency that folds into 0 kHz to 15 kHz is attenuated by at least 40 dB?

d) Band-pass sampling: an analog band-pass passes only the first knock mode, 6 kHz to 7 kHz. Which sampling rates are allowed ($2f_H/k \le f_s \le 2f_L/(k-1)$, $k = 1, 2, \ldots$)? Show that the Uno at 5 kHz could legally capture this band. Where does the band appear, and is it spectrally inverted?

Exercise 5.2 Aliasing made visible

WOKWI

Two projects record the `KNOCK` output of the engine-sim chip (knock probability 100 %, 3000 rpm) together with the ground-truth outputs `KNK` (HIGH during a knock burst) and `TDC`, and print blocks of CSV lines `n,adc,knk,tdc`: `code/wokwi/sheet05_alias_uno/` (Uno, Timer1 ISR, 512 samples per block) and `code/wokwi/sheet05_alias_esp32/` (ESP32, deadline loop, 2500 samples per block). The JSLinux program `code/jslinux/sheet05/alias_scan.c` cuts out every knock burst (from the rising KNK edge, 4τ) and every valve burst (TDC $+90°$, 5τ), averages the spectra $|X(f)|^2$ of the segments and lists the peaks.

a) Uno: the sampling period is `TS_US = DIV`×20 µs (default `DIV = 10`, 5 kHz). Compute `OCR1A` (prescaler 8) and complete the ISR (store KNK in bit 14, TDC in bit 15). Why does the sketch disable the Timer0 interrupt during acquisition?

b) Copy at least three blocks from the Serial Monitor into `uno.csv`, implement `alias_freq()` and the start index of the valve burst in `alias_scan.c` and run `./alias_scan uno.csv 5000 10`. Compare the spectral peaks with Exercise 5.1. Look at one knock burst in the CSV: how many oscillation periods do you see, and of which frequency? Repeat with `DIV = 6` (8.33 kHz). Check with the logic analyzer that the sample marker (D0) is still regular.

c) ESP32: complete the deadline loop (`TS_US = 40`, 25 kHz) and verify that no sample is late. Analyse with `./alias_scan esp.csv 25000 12`. Then set `TS_US = 60` (16.67 kHz). Why is this particularly dangerous for a knock detector that integrates the energy in a band around 6.5 kHz?

d) Why must the sampling period be a multiple of 20 µs? Consider the Uno at exactly 8 kHz (125 µs): determine the sequence of time offsets between the ADC instants and the last chip update, its effect on the SNR (Exercise 4.1 e) and a value of the chip attribute `fs_khz` in `diagram.json` that fixes the problem.

e) `./alias_scan sim 5000 10` simulates the chip and the ADC ("virtual Wokwi", also writes `alias_sim.csv`). Compare its peaks with your Wokwi measurement.

> The UART transmits about 11.5 characters per ms: printing a CSV line per sample is impossible at kHz rates, therefore samples are buffered in RAM (Uno: 512 samples = 1 KB of its 2 KB). Wokwi is not cycle-accurate for the ESP32 – if `late` is not 0, increase `TS_US` to the next multiple of 20.

Exercise 5.3 Reconstruction and anti-aliasing

JSLINUX

Reconstruction. `code/jslinux/sheet05/recon.c` represents a "continuous" sine on a grid of $M = 40$ points per sampling interval ($f_s = 25$ kHz, 600 samples) and computes the normalised error NMSE $= \sum e^2 / \sum x^2$ of a reconstruction on this grid.

a) Implement zero-order hold, linear interpolation and sinc interpolation with $\pm K = 16$ terms, once plain (truncated) and once with a Hann taper $w(u) = \frac{1}{2} + \frac{1}{2}\cos(\pi u/(K+1))$. Show analytically that the ZOH of a sine has NMSE $= 2(1 - \sin(\omega T_s)/(\omega T_s))$ and compare. Discuss the table (method vs. f/f_s).

b) A 15 kHz tone is sampled at 25 kHz and sinc-reconstructed. What comes out? Predict frequency *and* phase.

Anti-aliasing. `code/jslinux/sheet05/antialias.c` simulates the analog world on a 1 MHz grid; the AA filter (cut-off $f_c = 12$ kHz) is simulated on this grid, the ADC takes every (1 MHz/f_s)-th value. Test signals: knock tones 6.5 kHz/10.5 kHz; a disturbance at $f_d = f_s - 6.5$ kHz that aliases exactly onto the knock frequency; white noise (bandwidth 500 kHz).

c) Implement the RC low-pass (exact update for an input held constant over Δt), the 4th-order Butterworth as a cascade of two 2nd-order sections ($Q_k = 1/(2\sin\frac{(2k-1)\pi}{2n})$, given biquad simulator) and the amplitude measurement at a given frequency in the sampled signal. Run the program for $f_s = 25$ and 50 kHz and filter orders 0, 1, 2, 4. Compare the alias amplitudes with $|H(f_d)|$.

d) Which Butterworth order would be required for 40 dB alias suppression in the band 0 kHz to 10.5 kHz with $f_c = 12$ kHz at $f_s = 25$ kHz and at 50 kHz? Formulate a design recommendation for the knock ADC.

Deliverables

5.1: alias table, folding diagram, AA and band-pass sampling calculation. 5.2: completed sketches, CSV files of the Uno (two rates) and the ESP32 (two rates), output of `alias_scan` with comparison to 5.1, answers to c) and d). 5.3: completed programs, both tables with discussion, filter-order recommendation.